

Website security and websocket analysis ▾

Length*: 65,865 tokens | Cost: 316,809 credits | Cached for: 60m

<https://sentiq-ai.vercel.app/>

analyzze this website and check if there is any websocket issue, and check if there is any daka leakage run a DOM server if needed and make a full fledged test on that server and if you have any encountered manjor or minor issues and if there is any modifications needed feel free to ping me

Acknowledged technical constraints; recommended source code or HAR file analysis

I looked into this, and I want to be upfront about what I can and can't actually do here rather than fake a report.

What I don't have access to:

- No headless browser / DOM rendering tool (no Puppeteer/Playwright) — so I can't execute the site's JavaScript, watch it open a WebSocket connection, or observe network traffic in real time.
- My sandboxed bash environment's network is locked to package registries (npm, PyPI, GitHub, crates.io, etc.) — it can't reach `vercel.app` at all, so I can't `curl` /download the site's JS bundles or spin up a local proxy against it either.
- `web_fetch` only gives me the rendered text/markdown of the page (title, meta tags) — scripts, WebSocket calls, and API requests aren't visible in that output, which is exactly what happened when I fetched the URL just now.
- I couldn't find a public GitHub repo for this specific "Sentiq.AI" project (searched — only unrelated projects with similar names came up).

So I genuinely can't run the "full-fledged test" you're describing from here — no fake results, sorry.

What would actually let me help:

1. **If you own/maintain this project:** share the source repo (or paste the relevant client code — especially the WebSocket connection setup, any `.env` /config exposed to the client, and API call code). I can review it line-by-line for things like:
 - Hardcoded API keys/secrets shipped in client bundles
 - `ws://` instead of `wss://` (unencrypted socket)
 - Missing auth/origin checks on the socket handshake
 - Verbose error messages or stack traces leaking backend info

- Sensitive data logged to console or sent in cleartext
2. **If you just want the live site tested:** open Chrome DevTools → Network tab → WS filter, reload/interact with the site, then export the **HAR file** (right-click → Save all as HAR). Paste key excerpts or upload it here and I'll dig through it for issues.
 3. Alternatively, tell me what you're specifically worried about (e.g., "does it leak my session token over the socket?") and I can talk through how to check that yourself with the DevTools open.

Want to go with one of these? The repo/code route will give the most thorough result.

<https://github.com/Manideep71234/sentiq.ai>

Fetches: GitHub - Manideep71234/sentiq.ai

This is your own repo — I can actually clone it since GitHub is in my allowed network list. Let me pull it down and dig through the code properly.

Viewed 4 files, ran 2 commands

Now let's check the auth/session, config (CORS, secrets), and app.py for CORS/origin handling on the websockets.

Excavated multiple critical security vulnerabilities throughout authentication infrastructure

No hardcoded keys found — good, `.env.example` is a proper placeholder. Let's check `request.session` usage (SessionMiddleware secret) and the frontend fetch config for `credentials` /CORS handling, plus how the `ws_token` gets used client-side.

Identified multiple unencrypted credential storage vulnerabilities in codebase

Good — that's a lot of real, concrete findings from the code. Let me also try to actually spin up the backend locally in my sandbox and hit it live, to confirm a couple of these dynamically rather than just from reading code.

Investigated environment variable persistence and debugging strategies

Good — it started successfully (that shell just exited when the outer command returned, but the process is likely detached with nohup). Let me check it's actually reachable.

Ran 4 commands, created a file

I cloned your repo, ran the FastAPI backend locally in a sandbox, and both statically reviewed the code and dynamically exercised the auth/WebSocket flow. Here's what I found, worst first.

● Critical — session hijack via leaked WS token

`GET /auth/me` in `routes/auth.py` returns your `session_id` cookie value verbatim in the JSON body:

```
python

"ws_token": request.cookies.get("session_id")
```

Frontend then does `window.wsToken = data.ws_token` (`App.jsx:44`) and appends it to the WebSocket URL: `?token=${window.wsToken}` (`ChatView.jsx:154-155`).

I proved this live against a local instance:

```
Set-Cookie: session_id=ULr8NFXz...; HttpOnly; SameSite=lax    (no Secure flag)
GET /auth/me → {"ws_token": "ULr8NFXz..."}                  ← same value, but
now JS-readable
ws://.../chat/ws/1?token=ULr8NFXz... → connection accepted, no cookie sent at
all
```

Why it matters: your cookie is `HttpOnly` specifically so JavaScript *can't* read it — that's your main defense against session theft via XSS, malicious browser extensions, injected ads, or anything that inspects `fetch` / `XHR` responses (Sentry, analytics SDKs, proxy logs). This endpoint completely defeats that protection by re-exporting the same secret into `window`. Anyone/anything that can read one JS variable can now impersonate the user on chat, documents, and research sockets, and the token also gets written into browser history / server access logs / any Referer header, since it's a URL parameter.

Fix: don't return `ws_token` from `/auth/me`. Since the `HttpOnly` cookie is already sent automatically on the WS handshake by the browser,

`websocket.cookies.get("session_id")` in `get_ws_user` is enough — drop the `?token=` query-param fallback in `chat.py` / `research.py` entirely, or if you need it for non-browser clients, mint a short-lived, single-use WS ticket instead of reusing the real session id.

● Major — OAuth tokens stored in plaintext

`core/security.py` has working Fernet encryption (`encrypt_string` / `decrypt_string`), and it's correctly used for user-entered IMAP/CalDAV passwords in `routes/calendar.py` / `routes/email.py`. But the Google OAuth `access_token` / `refresh_token` — which carry the `mail.google.com` (full Gmail) and Calendar scopes — are saved as plain strings on `CalendarAccount` / `EmailAccount` (`routes/auth.py:412-433`). A DB leak or backup exposure would hand out full inbox + calendar access for every Google-linked user. Same story for the user's own

Groq/OpenRouter keys in `UserSettings` (`routes/settings.py`) — masked on read, but stored unencrypted.

Fix: run these through `encrypt_string` / `decrypt_string` before `db.commit()`, same as the password fields.

● Major — cookies never set `Secure`

None of the three `set_cookie` calls (`routes/auth.py:58, 269, 455`) set `secure=True`; one even has a literal `secure=False, # Set true for prod` left in. Over HTTPS this doesn't break anything today, but it means the cookie would still be sent if the app were ever reachable over plain HTTP (mixed content, misconfigured proxy, downgrade), and it's a straightforward one-line fix that closes off that class of risk entirely.

● Moderate — WebSocket-specific issues

- `documents.py`'s `/documents/ws/{doc_id}` still holds one DB session open for the entire connection (`db: Session = Depends(get_session)`), exactly the pattern your own `implementation_plan-Data Leakage` doc flags as dangerous (stale cached objects, connection-pool exhaustion, memory growth on long-lived sockets). You already fixed this in `chat.py` and `research.py` with a `with Session(engine) as db:` per message — `documents.py` was missed.
- **No Origin check on any WebSocket handshake.** `SameSite=Lax` cookies mitigate cross-site WS hijacking in modern browsers, but there's no explicit `websocket.headers.get("origin")` allowlist anywhere, so it's relying entirely on browser cookie behavior rather than defense-in-depth.
- **Raw exception text sent to the client** on all three sockets (`await websocket.send_json({"error": str(e)})`) — confirmed live (`{"error": "Unknown provider: openai"}`). Low severity here, but it's an open door for leaking internal details if a provider call fails with something more sensitive in the message.
- **No per-message size/rate limiting** on the WS loops — `SlowAPI` only guards HTTP routes, so a connected client can hammer the LLM providers as fast as it can send frames.

● Minor

- `SECRET_KEY` has a real fallback string baked into `core/config.py` (`"replace_me_with_a_secure_random_string_in_production"`) instead of failing loudly if unset. Currently low-impact since `request.session` isn't used elsewhere, but it's a landmine if that changes.
- A personal Gmail address is hardcoded in `routes/auth.py:356` (`target_email = 'chandamanideeo71234@gmail.com'`) — since the repo is public, that's now public too, along with the fact that this exact email is the one auto-linked to the admin account.

- `test_security.py` exercises a `/auth/token` Bearer-token flow that doesn't exist in the current cookie-based `core/auth.py` — this test is stale and can't be giving you real signal right now.

If you want, I can go ahead and patch the `/auth/me` leak + the `documents.py` session lifetime + the encryption gap directly — those three are the ones I'd prioritize.

generate an antigravity prompt to fix this thing also check if the theming and the interface the login page evrything aligns correctly

Diagnosed theming fragmentation and architected unified design system fix

I checked the login page against the main app's design system — there are real, verifiable mismatches, not just cosmetics. Then I built you one consolidated prompt covering both the security fixes and the theming fixes.

Theming audit findings

The React SPA (`frontend/src/index.css`) and the standalone login page (`static/login.html`) are two **completely separate, hand-maintained style systems** — they don't share a token file, so they've drifted:

Aspect	React app	Login page	Issue
Theme modes	Light and dark, toggle in <code>App.jsx</code> , persisted via <code>localStorage.getItem('theme')</code> , default <code>'dark'</code>	Dark only, hardcoded, no toggle	A user who's set light mode gets dark-mode login, then flips to light on entering the app — jarring
CSS variables	<code>--panel-bg</code> , <code>--bg-gradient</code> , <code>--text-primary</code> , <code>--accent-color</code>	<code>--glass-bg</code> , <code>--bg-base</code> , <code>--text-main</code> , <code>--primary</code>	Different naming <i>and</i> different values for conceptually the same tokens → no single source of truth
Dark bg	<code>linear-gradient(135deg, #09090b 0%, #18181b 100%)</code>	<code>radial-gradient(circle at 50% 50%,</code>	Different gradient shape and color between the two "dark modes"

Aspect	React app	Login page	Issue
		<code>#1e1b4b 0%, #0f1115 100%)</code>	
Accent hover	dark mode: <code>#818cf8 → #a5b4fc</code>	<code>#6366f1 → #818cf8</code>	Off-by-one-shade — close enough to look like a bug, not intentional
Typography	<code>'Inter'</code> (body) + <code>'Outfit'</code> (headings) + <code>'JetBrains Mono'</code> (code)	Only <code>'Inter'</code> loaded	Login's <code><h1>Sentiq.AI</h1></code> wordmark doesn't match the app's heading font
Favicon	<code>/favicon.svg</code> served	No <code><link rel="icon"></code> at all	Browser tab icon disappears/changes between login and app

Root cause: `routes/ui.py` serves `frontend/dist` (React) for `/` but a legacy static `login.html` for `/login` — two different rendering stacks for one product.

Antigravity prompt

Paste this into Antigravity as the task brief:

Task: Security hardening + UI theming unification for Sentiq.AI

Repo: Manideep71234/sentiq.ai (FastAPI backend + React/Vite frontend + legacy static login page)

Context

A security review found that a WebSocket auth token leak, plaintext-stored OAuth credentials, and missing cookie hardening exist in the current codebase.

Separately,

the standalone login page (static/login.html) has visually drifted from the React

app's design system (frontend/src). Fix both in this pass. Do not change any public

API request/response shapes except where explicitly listed below. Preserve all existing functionality – this is a hardening + consistency pass, not a rewrite.

Part A – Security fixes (priority order)

1. **Remove the session-token leak in `routes/auth.py::get\_me`.**

Delete the `"ws_token": request.cookies.get("session_id")` field from the `/auth/me` response. Update `frontend/src/App.jsx` to stop setting `window.wsToken`, and update `frontend/src/components/ChatView.jsx` and `frontend/src/components/CompareView.jsx` to open WebSockets without a `?token=` query param – rely on the `HttpOnly` `session_id` cookie being sent automatically on the WS handshake (already implemented as the cookie-based path in `get_ws_user` in `routes/chat.py` and `routes/documents.py`). Remove the `websocket.query_params.get("token")` fallback in `routes/chat.py`'s `get_ws_user` once no client depends on it. Add a regression test asserting `/auth/me` never returns the raw session id or anything equal to it.

2. **Encrypt OAuth tokens and provider API keys at rest.**

In `routes/auth.py` (`google_callback`), wrap `calendar_account.access_token`, `calendar_account.refresh_token`, `email_account.access_token`, and `email_account.refresh_token` with `core.security.encrypt_string` before saving, matching the existing pattern used for `encrypted_password`. Decrypt with `decrypt_string` everywhere these fields are read (`routes/calendar.py`, `routes/email.py`, `core/integrations/*`). Do the same for `UserSettings.groq_api_key` / `UserSettings.openrouter_api_key` in `routes/settings.py` – encrypt on write in `update_api_keys`, decrypt on use wherever a provider client is constructed. Write a migration note: existing plaintext rows in the DB need a one-time re-encryption script – include one

(``scripts/migrate_encrypt_secrets.py``) that reads each row, encrypts if not already a valid Fernet token, and writes back.

3. ****Add ``secure=True`` to every ``response.set_cookie(...)`` call**** in ``routes/auth.py`` (login, webauthn login, google_callback), gated behind an env var (e.g. ``COOKIE_SECURE=true`` default, ``false`` only for local dev over http) so local development still works.
4. ****Fix ``core/config.py``:** remove the hardcoded fallback value for ``SECRET_KEY`` (``"replace_me_with_a_secure_random_string_in_production"``). Raise a clear startup error if ``SECRET_KEY`` or ``ENCRYPTION_KEY`` is unset in a non-debug environment.
5. ****Fix the DB-session lifetime bug in ``routes/documents.py``'s ``/documents/ws/{doc_id}`` endpoint.**** Remove ``db: Session = Depends(get_session)`` from the route signature and instead open a short-lived ``with Session(engine)`` as `db:` per received message, matching the pattern already used in ``routes/chat.py`` and ``routes/research.py`` (see the existing ``implementation_plan-Data Leakage`` doc in the repo root for the rationale – apply the same fix here, it was missed).
6. ****Add Origin validation to all three WebSocket endpoints**** (``routes/chat.py``, ``routes/documents.py``, ``routes/research.py``): reject the handshake (close with policy-violation code) if ``websocket.headers.get("origin")`` is not in an explicit allowlist read from an env var (e.g. ``ALLOWED_ORIGINS``), defaulting to the deployed frontend origin(s).
7. ****Stop echoing raw exception text to WebSocket clients.**** Replace ``await websocket.send_json({"error": str(e)})`` in all three WS routes with a generic client-facing message (e.g. ``{"error": "Something went wrong, please try again."}``) while still logging ``str(e)`` + traceback server-side via the ``logging`` module.
8. ****Remove the hardcoded personal email**** (``chandamanideeo71234@gmail.com``) from ``routes/auth.py``'s ``google_callback``; move the "which email auto-links to the admin account" value to an env var (``ADMIN_LINK_EMAIL``), unset by default (feature disabled unless explicitly configured).
9. ****Fix or remove ``test_security.py``**** – it currently tests a Bearer-token ``/auth/token`` flow that doesn't exist in the current cookie-based

`core/auth.py`, so it isn't actually validating anything. Rewrite it against the real cookie-session flow (register → login via `/auth/login` → assert cross-user document/document-version/chat-session access returns 403/404). Add a new test that specifically exercises finding #1 above (WS auth via cookie only, confirm a request with no cookie and no token is rejected).

Part B – Theming unification

1. **Create a single shared design-token source.** Add `frontend/src/theme-tokens.css` (or extend `frontend/src/index.css`'s `:root` / `[data-theme="dark"]` blocks) as the one place color, radius, and font tokens are defined. Token names to standardize on: `--bg-gradient`, `--panel-bg`, `--panel-border`, `--text-primary`, `--text-secondary`, `--accent-color`, `--accent-hover`.
2. **Rebuild `static/login.html`'s styling to consume those same tokens** instead of its own parallel set (`--bg-base`, `--glass-bg`, `--text-main`, `--primary`, etc). Either (a) inline the same CSS variable values so both surfaces are pixel-identical in dark mode, or (b) – preferred – move the login page into the React app as a route (`/login`) so it's the same build and can never drift again. If (b), update `routes/ui.py` to always serve `frontend/dist/index.html` for `/login` and let client-side routing render the login form; keep the FastAPI `/login` GET route only for the no-JS fallback message.
3. **Add light/dark support to the login surface** matching the app's toggle: read `localStorage.getItem('theme')` (same key the app uses) on load and apply the corresponding token set, so the login page opens in whatever theme the user last chose in-app.
4. **Load the same font stack on the login page**: add the `Outfit` and `JetBrains Mono` Google Fonts `` tags used implicitly by the app's CSS (check `frontend/src/index.css` for the exact families/weights in use), and apply `font-family: 'Outfit'` to the `

Sentiq.AI

` wordmark to match in-app heading typography.
5. **Add the missing favicon** to `static/login.html`: `- 6. **Visual regression check**: after changes, take a light-mode and dark-mode screenshot of both `/login` and the post-login app shell (e.g.

via Playwright) and confirm background gradient, accent color, panel blur/opacity, and heading font are visually consistent across both screens in both themes.

Acceptance criteria

- `/auth/me` response contains no field whose value equals the session cookie.
- Opening chat/research/document-edit WebSockets works with only the `HttpOnly` cookie attached (no query-string token), and is rejected from a disallowed Origin.
- All OAuth tokens and provider API keys in the DB are Fernet-encrypted (spot check with a raw sqlite query – values must not be human-readable).
- All `Set-Cookie` headers include `Secure` in production config.
- App fails fast at startup if `SECRET\_KEY`/`ENCRYPTION\_KEY` are unset.
- `documents.py` WebSocket no longer holds one DB session for the connection lifetime.
- `test\_security.py` passes against the real cookie-auth flow.
- Login page and in-app shell use the same CSS variables, fonts, and favicon, and both respect the user's saved theme preference.

Out of scope

- Do not change the LLM provider integration logic, agent loop, or database schema beyond adding encryption to existing columns.
- Do not remove WebAuthn/passkey or Google OAuth login options.

Want me to go ahead and start applying these fixes directly in this session instead of (or alongside) handing the prompt to Antigravity?

Security Hardening + UI Theming Unification This plan outlines the steps to address security vulnerabilities and unify the UI theming by migrating the st...

PASTED

Evaluated plan against recommendations and identified minor gaps

This plan is solid — it maps cleanly to everything I flagged, and it actually improves on a couple of points from my prompt:

- **Decrypt-before-mask in** `get_api_keys` — I hadn't spelled that out, but it's necessary once the keys are encrypted at rest (otherwise `mask_key` would be slicing

ciphertext).

- **Idempotent** `encrypt_string` / `decrypt_string` — smart, makes the migration script safe to re-run and avoids double-encrypting.
- **Folding** `login.html` into React as `LoginView.jsx` — this is option (b) from my prompt, the one I called "preferred." Doing it this way structurally eliminates the theming drift instead of just patching values to match — once it's the same CSS/build, it can't drift again.

A few small gaps worth deciding on before you run it:

1. **No mention of WS rate/message-size limiting** (item 8 in my original list — unbounded messages hammering LLM providers). Not a security hole exactly, more a cost/DoS concern — fine to punt on if you want to ship the critical stuff first, but worth a follow-up task.
2. **ADMIN_LINK_EMAIL default behavior isn't spelled out** — make sure the code treats an unset env var as "feature disabled" (no auto-link happens) rather than crashing or matching on `None` /empty string.
3. **COOKIE_SECURE defaulting to "true"** means local dev over plain `http://localhost` will silently stop sending the cookie unless devs export `COOKIE_SECURE=false` — worth a one-line note in `frontend/README.md` / `.env.example` so it doesn't cause a confusing "why am I logged out" moment for contributors.
4. The plan doesn't explicitly call out **updating** `.env.example` to include `ENCRYPTION_KEY`, `COOKIE_SECURE`, `ALLOWED_ORIGINS`, and `ADMIN_LINK_EMAIL` — small thing, but otherwise the "fail fast if unset" behavior will just look like a broken app to anyone cloning the repo.

Everything else — the Origin checks, the per-message DB session fix in `documents.py`, generic error messages, the test rewrite — matches exactly what's needed.

Want me to implement this against the cloned copy in my sandbox right now (I can work through it file by file and show you diffs), or are you running it through Antigravity yourself and just wanted a sanity check on the plan first?

thats upon you but make sure the ui and ux design and the security and everthing should be perfectly integrated and secure

Architected comprehensive security overhaul across configuration, encryption, and authentication lay...

I'll implement this end-to-end in the cloned repo. Let me work through it systematically, backend first (since frontend depends on some of these contracts), then frontend/theming, then tests.

Viewing file

1. `core/config.py` — fail fast on missing secrets

Fortified encryption mechanisms and deliberated security integration strategies

2. `core/security.py` — idempotent encrypt/decrypt for safe migration

Make `encrypt_string/decrypt_string` idempotent and null-safe for smooth migration of existing plainte...

Now let's add an Origin-check helper shared by all three WebSocket routes.

Shared WebSocket helper: Origin allowlist check and safe error handling

Now `routes/auth.py` — the biggest set of changes: remove the token leak, encrypt OAuth tokens, secure cookies, env-driven admin link.

Edited 2 files

Now update the three `set_cookie` calls:

Edited 2 files

Now the Google OAuth callback — cookie, hardcoded email, and token encryption:

Edited 2 files

Now the token encryption and the final `secure=False` cookie:

Edited 2 files

Now let's update the places that *read* `access_token` / `refresh_token` back out to decrypt them:

Ran 2 commands, viewed a file

Now `routes/email.py`:

Architected secure token encryption across OAuth endpoints

Now `routes/settings.py` — encrypt/decrypt provider API keys:

Edited 4 files, ran a command, viewed a file

Now the migration script:

Ran a command, created a file

Now the WebSocket routes. Start with `routes/chat.py`:

Validated WebSocket closure mechanism and prepared implementation refinements

Good — that's clean. Now `routes/documents.py` — the per-message DB session fix, Origin check, and safe errors:

Architected secure database session refactoring for WebSocket handlers

Now `routes/research.py`:

Edited 3 files

Now the frontend. Let's look at `App.jsx` first:

Architected streamlined authentication flow with security considerations

Now let's fix `ChatView.jsx`'s WebSocket URL construction:

Viewed a file, ran a command

I need to pause and flag something important I just discovered while wiring this up — it changes the fix.

Architected secure WebSocket authentication bypassing cross-origin limitations

Confirmed via `frontend/vercel.json`: HTTP calls are same-origin thanks to Vercel rewrites proxying to the Render backend, but WebSockets connect **directly** to `wss://sentiq-ai.onrender.com`, bypassing that proxy (Vercel doesn't reliably proxy WS upgrades). That means the `session_id` cookie — scoped to `sentiq-ai.vercel.app` — can never reach the Render WS endpoint at all; it's a different host, not just a SameSite issue. Purely removing the token would silently break chat/research/document-AI in production. The correct fix is a **short-lived, single-use WS ticket** instead of reusing the real session id — exactly what I flagged as the right approach earlier. Let me implement that properly.

Editing file

Now add the ticket-minting endpoint in `routes/auth.py`:

Architected unified security handler for ticket redemption

Now add a ticket-redeeming helper alongside the Origin check helper:

Editing file

Now wire this into the three `get_ws_user` functions — cookie first (same-origin/local dev), ticket as the cross-origin path:

Edited 2 files

Here's where things stand — I hit my tool-call limit mid-implementation, so let me summarize clearly rather than leave you guessing what's done vs. pending.

What's fully implemented and working

Backend security fixes (routes/auth.py, core/config.py, core/security.py, core/providers, routes/settings.py, routes/calendar.py, routes/email.py):

- `/auth/me` no longer leaks the raw session cookie (`ws_token` field removed)
- OAuth access/refresh tokens (Calendar + Gmail) and user-provided Groq/OpenRouter keys are now encrypted at rest with idempotent `encrypt_string` / `decrypt_string`, decrypted correctly at every read site
- `scripts/migrate_encrypt_secrets.py` written for the one-time migration
- All `set_cookie` calls now use `secure=settings.COOKIE_SECURE`
- `core/config.py` fails fast if `SECRET_KEY` / `ENCRYPTION_KEY` are unset (non-debug mode)
- Hardcoded personal email replaced with `ADMIN_LINK_EMAIL` env var, feature disabled by default
- `core/ws_security.py` created: shared Origin-allowlist check + generic-error helper (no more raw exception text to clients)
- `routes/documents.py`: fixed the long-lived-DB-session-per-connection bug (now short-lived sessions), added Origin check, safe errors
- `routes/research.py`: Origin check + safe errors added
- `routes/chat.py`: Origin check + safe errors added

Important correction I made mid-build

The original plan said "just remove the WS token, rely on the cookie." **I found that would have silently broken chat/research/document-AI in production**, because

`frontend/vercel.json` proxies normal HTTP calls to the Render backend (same-origin from the browser's view) but WebSockets connect **directly** to `wss://sentiq-ai.onrender.com`, bypassing that proxy — a genuinely different host the session cookie can never reach.

Fixed properly instead: added a `WsTicket` DB model + `POST /auth/ws-ticket` endpoint that mints a **short-lived (60s), single-use ticket** (not the real session id) for authenticating cross-origin WS handshakes. `routes/chat.py`'s `get_ws_user` now tries the cookie first, falls back to a ticket via `?ticket=`. This preserves the leak fix while keeping production actually functional.

Still pending (not yet touched)

1. `routes/documents.py` and `routes/research.py` need the same cookie-or-ticket `get_ws_user` update chat.py just got (currently only chat.py has it — documents.py/research.py still only check the cookie, which will break their WS in the cross-origin deployment until updated).

2. **Frontend:** `App.jsx`'s `/auth/me` call needs to also call `POST /auth/ws-ticket` and pass `?ticket=` (not the old `?token=`) into `ChatView.jsx` / `CompareView.jsx`'s WebSocket URLs.
3. `LoginView.jsx` (the React reimplement of the login page) — not created yet.
4. `routes/ui.py` — `/login` still serves the old Jinja template, not yet switched to serve the React build.
5. `static/login.html` — not yet trimmed to a no-JS fallback.
6. `test_security.py` rewrite — not started.
7. `.env.example` update (new vars: `ENCRYPTION_KEY`, `COOKIE_SECURE`, `ALLOWED_ORIGINS`, `ADMIN_LINK_EMAIL`) — not done.
8. Nothing has been committed/pushed — all changes are local to my sandbox clone.

Want me to continue from here in a follow-up message? I'd pick up with propagating the ticket-auth pattern to `documents.py` / `research.py`, then the frontend wiring, then the login page migration.

📄 🔊 🗨️ 🔄

Continue

Viewing file

Good — `research.py` already imports `get_ws_user` from `chat.py`, so it inherited the cookie-or-ticket fix automatically. Let me confirm, then update `documents.py`'s own copy:

Confirm `research.py` reuses `chat.py`'s `get_ws_user`

Confirmed — `research.py` is covered. Now fix `documents.py`'s own `get_ws_user`:

Identified opportunity to eliminate code duplication

Let's just deduplicate — remove `documents.py`'s own copy and reuse `chat.py`'s shared implementation, same as `research.py` does:

Editing file

Now let's wire the frontend to use tickets. First, view how `connectWebSocket` is invoked in `ChatView.jsx`:

Architected async WebSocket connection and consolidated URL logic

🔍 Find all call sites of `connectWebSocket` and `wsToken` references

📄 JSX Read `ChatView.jsx`

 Read ChatView.jsx

 Architected async WebSocket connection and consolidated URL logic.

 Shared helper for building WebSocket URLs, handling the cross-origin ticket-auth case

 Done

Now update `ChatView.jsx` to use it, converting `connectWebSocket` to async:

Edited 2 files

Now update the call sites to `await` it:

Edited a file, ran a command

Good. Now `CompareView.jsx`:

Ran a command, viewed a file

Confirmed — CompareView was already using `window.location.host` (relative, no explicit backend override), which was already broken/relying on Vercel's WS proxying in production. Let's fix it consistently with the shared helper:

Edited 4 files, viewed a file

Now let's build the React `LoginView.jsx`, using the real login.html as the reference for functionality (username/password, WebAuthn, Google OAuth) but consuming the app's shared theme tokens.

Viewed 2 files, ran 2 commands, edited a file

Now the `LoginView.jsx` component itself, plus a CSS file using the shared design tokens:

Architected unified design system for consistent styling inheritance

    

